

Resumen del capítulo: data pipelines y por qué utilizarlos

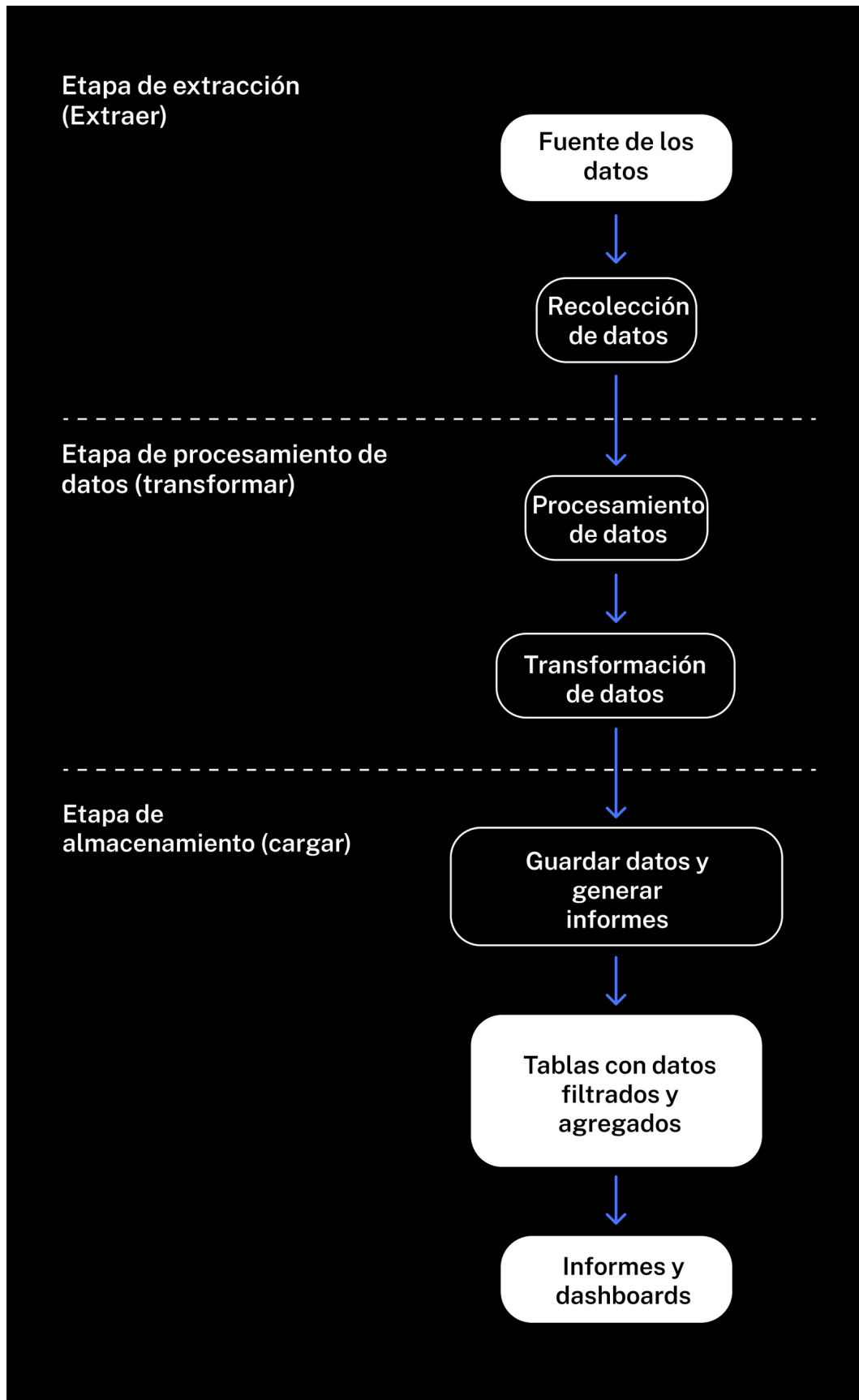
Los pipelines de datos son la base de la automatización. Son programas especiales que se ejecutan según horarios. Recopilan, fusionan, transforman y almacenan datos automáticamente.

Los pipelines permiten:

- Analizar datos en Internet y almacenar los resultados en una base de datos
- Recopilar información sobre las visitas y compras de varios usuarios de los sistemas corporativos y crear informes para un mayor análisis de cohortes
- Detectar anomalías en el comportamiento del usuario
- Analizar pruebas A/B
- Enviar muchísimo spam a tus compañeros de equipo sobre el crecimiento de ventas y visitas

¡Y todo sin mover un dedo humano! Solo dedos de robot.

Las etapas de un pipeline se pueden describir con el acrónimo inglés **ETL** de "extract, transform, load" que significa extraer, transformar, cargar.



En la etapa de extracción, los pipelines recopilan datos de varias fuentes, como sitios web, bases de datos de la empresa y sus socios, y API externas.

En la etapa de transformación, los datos recopilados se estandarizan. Por ejemplo, el texto puede convertirse en números o fechas. Después se categoriza. Luego de la categorización, los datos se transforman o se convierten a un formato conveniente para crear informes o almacenar información agregada. Por ejemplo, en esta etapa, los datos sobre visitas y ventas se pueden transformar en tablas, donde se calcula el valor de tiempo de vida de un cliente (LTV) para un posible análisis de cohortes.

En la última etapa, cargar, los pipelines guardan los datos agregados en las tablas de la base de datos y crean informes y envíos por correo.

Las pipelines de datos están diseñados de tal manera que todas sus etapas se pueden lanzar una y otra vez, produciendo resultados consistentes.

Agregar datos y crear tablas en bases de datos

Los datos de origen para el análisis normalmente se almacenan en bases de datos. Por lo general, son **registros sin procesar** (registros de eventos).

Afortunadamente, los y las analistas rara vez necesitan datos sin procesar, ya que los datos agregados son suficientes para los informes y conclusiones. La **agregación** es el proceso de agrupar datos y reducir su tamaño.

¡Leer millones de registros de una base de datos puede llevar horas! Es mejor hacer una nueva tabla donde almacenarás los datos agrupados por mes y país.

Para almacenar datos agregados, debes crear una tabla en psql mediante el comando **CREATE TABLE**. Esta es su sintaxis:

```
CREATE TABLE table_name (primary_key data_type,  
                           column_name1 data_type,  
                           column_name2 data_type,  
                           ...);
```

PostgreSQL tiene muchos tipos de datos diferentes. Necesitaremos los básicos:

- **INT** — número entero
- **REAL** — número de coma flotante
- **VARCHAR(n)** — una string, donde n es la longitud máxima. Por ejemplo, `VARCHAR(128)` es para un campo que contiene una string de no más de 128 caracteres
- **TIMESTAMP** — fecha y hora
- **SERIAL** — un tipo de datos especial para valores de clave primaria. Recuerda que una clave primaria es el número único de un registro de tabla. La clave primaria se define con la siguiente expresión:

```
CREATE TABLE table_name (primary_key_field_name serial PRIMARY KEY);
```

Cuando agregas un nuevo registro a la tabla, el SGBD le asigna un número de fila que es 1 mayor que el anterior y coloca este número en un campo con el tipo SERIAL.

Tablas verticales y horizontales

Al diseñar tablas agregadas, intenta siempre que sean verticales, lo que significa que cada elemento de los datos tiene su propia columna. Cuando las cosas se organizan de esta manera, la automatización y el escalado se vuelven mucho más fáciles.

El uso de tablas horizontales genera problemas: cada vez que agregas una nueva columna al pipeline, así como a los dashboards e informes conectados a él, tendrás que definir el tipo de columna y las reglas para procesar los valores ausentes. Puedes hacerlo en el código, por supuesto, pero es mucho más simple crear una tabla del primer tipo. No tendrás que redefinir sus estructuras si decides agregar nuevos datos.

Algo puede salir mal al crear una tabla. Puede resultar, por ejemplo, que la tabla necesite una estructura diferente. ¡No te preocupes! Simplemente elimínala en psql con el comando **DROP TABLE**:

```
DROP TABLE table_name;
```

Una vez que hayas eliminado las tablas incorrectas y dejado las que necesitas, es hora de otorgar acceso a ellas en psql. Para hacerlo, usa el comando **GRANT**:

```
GRANT ALL PRIVILEGES ON TABLE agg_games_year TO anthony;  
-- aquí anthony es el nombre de un usuario que quiere acceder a la tabla
```

- **TO** define el usuario al que se le dará permiso
- **ON TABLE** indica la tabla a la que se debe proporcionar acceso
- **ALL PRIVILEGES** significa que el usuario obtiene todos los permisos posibles para trabajar con la tabla: puede leer, escribir y eliminar datos

Además de los derechos de acceso, también debes proporcionar permisos para trabajar con claves primarias. Cuando creas un campo SERIAL con claves primarias, el SGBD crea automáticamente un objeto **SEQUENCE**, que almacena datos sobre la forma en la que se deben generar los identificadores de claves primarias. Estos objetos reciben automáticamente nombres como este: `tableName_keyName_seq`.

```
GRANT USAGE, SELECT ON SEQUENCE table_name_id_seq TO anthony;
```

`GRANT USAGE` significa que el usuario ahora puede agregar nuevos valores a `agg_games_year_record_id_seq`. `GRANT SELECT` permite al usuario leer datos de la tabla.

Crear un script de pipeline

Aunque tus compañeros de equipo siempre pueden proporcionarte archivos de volcado, siempre tienes que intentar obtener acceso directo a las bases de datos. ¿Por qué?

- Esto acelerará tu trabajo, ya que no tendrás que esperar hasta que esté listo un volcado.
- La automatización es imposible sin acceso directo a los datos. Si tu colega se enferma o se va de vacaciones sin crear un archivo de volcado para ti, el proceso de automatización fallará.

- Puedes encontrar muchas cosas interesantes en las bases de datos.

La biblioteca **SQLAlchemy** te permite leer los datos de las tablas de las bases de datos en DataFrames de pandas y almacenar datos de pandas en una base de datos con solo un comando.

Nos conectaremos a la base de datos utilizando `sqlalchemy`. Vamos a crear un script en Sublime Text:

```
#!/usr/bin/python

# Importar librerías
import pandas as pd
from sqlalchemy import create_engine

# Definir los parámetros para conectarse a la base de datos
# los puedes solicitar al administrador de la base de datos.
db_config = {'user': 'my_user',          # nombre de usuario
             'pwd': 'my_user_password', # contraseña
             'host': 'localhost',        # dirección del servidor
             'port': 5432,                # puerto de conexión
             'db': 'games'}              # nombre de la base de datos

# Crear string de conexión de la base de datos.
connection_string = 'postgresql://{user}:{pwd}@{host}:{port}/{db}'.format(db_config['user'],
                                                                            db_config['pwd'],
                                                                            db_config['host'],
                                                                            db_config['port'],
                                                                            db_config['db'])

# Conectarse a la base de datos.
engine = create_engine(connection_string)

# Crear una consulta SQL.
query = ''' SELECT game_id, name, platform, year_of_release
            FROM data_raw
            ...

# Ejecutar la consulta y almacenar el resultado
# en el DataFrame.
# SQLAlchemy automáticamente dará a las columnas
# los mismos nombres que tienen en la tabla de la base de datos. Solo tendremos que
# especificar la columna de índice con index_col.
data_raw = pd.io.sql.read_sql(query, con = engine, index_col = 'game_id')

print(data_raw.head(5))
```

Estudiemos el código en detalle. Para conectarse a una base de datos, tienes que concretar:

- El SGBD
- La ubicación de la base de datos: la dirección IP del servidor donde se implementa la base de datos y el puerto que se conectará
- El nombre de usuario y contraseña para la conexión
- El nombre de la base de datos que necesitas (un servidor puede almacenar varias bases de datos)

SQLAlchemy analiza automáticamente si las claves primarias de una tabla de base de datos y los índices del DataFrame son los mismos. SQLAlchemy se guía por el parámetro `**if_exists**`. Si especificas `if_exists = 'replace'`, SQLAlchemy retirará los datos existentes de la tabla y guardará los datos nuevos. Si indicas `if_exists = 'append'`, SQLAlchemy agregará nuevas filas al final de la tabla.

```
df.to_sql(name = 'table_name', con = engine, if_exists = 'append', index = False))
```

A veces necesitarás eliminar filas antiguas de la tabla a medida que se ejecuta tu pipeline. Para ello, utiliza el comando especial de SQL **DELETE**:

```
DELETE FROM table_name WHERE conditions_for_finding_records_to_be_deleted;
```